



K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

School of Engineering & Technology



Foundations of Cyber Security Lab

(ETCYCS305/ CYS001)

B.Tech CSE (Cyber Security)/BCA/B.Sc

(2026-27)

Prepared By:

Arsalan Ahmed Alyas



Institution Vision & Mission

Vision

KR Mangalam University aspires to become an internationally recognized institution of higher learning through excellence in inter-disciplinary education, research and innovation, preparing socially responsible life-long learners contributing to nation building.

Mission

- Foster employability and entrepreneurship through futuristic curriculum and progressive pedagogy with cutting-edge technology.
- Instill notion of lifelong learning through stimulating research, Outcomes-based education and innovative thinking.
- Integrate global needs and expectations through collaborative programs with premier universities, research centers, industries and professional bodies.
- Enhance leadership qualities among the youth having understanding of ethical values and environmental realities.



School Vision & Mission

Vision

To excel in scientific and technical education through integrated teaching learning, research, and innovation.

Mission

- Creating a unique and innovative learning experience to enhance quality in the domain of Engineering & Technology.
- Promoting Curricular, co-curricular and extracurricular activities that support overall personality development and lifelong learning, emphasizing character building and ethical behaviour.
- Focusing on employability through research, innovation and entrepreneurial mindset development.
- Enhancing collaborations with National and International organizations and institutions to develop cross-cultural understanding to adapt and thrive in the 21st century.



1. Course Details

Programme Name	B.Tech CSE (Cyber Security)
Semester	III
Course Name	Foundations of Cyber Security
Course Code	CYS001
L-T-P	3-0-2
Credits	4
Contact Hours	45
Type of Course	DSE / SPC
Pre-requisite(s)	Basic understanding of computer networks, operating systems, and introductory scripting using Python or the Linux CLI.

2. Program Outcomes (POs)

Engineering Graduates will be able to:

1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.



7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

3. Lab Instructions & Course Outcomes (COs)

a. Lab Instructions

- Each lab is an analysis- and configuration-based activity and must be completed within the two-week submission window announced by the faculty.
- Students must submit the required evidence, lab record, screenshots/console traces, file upload or GitHub link on the ZOSIMA portal as instructed.
- All practical work must be performed strictly inside the isolated lab environment on systems the student owns or is explicitly authorised to test.
- Evidence must be original, clearly labelled, and demonstrated during lab/class or from home submission as permitted by the faculty. Every configuration change requires before-and-after proof.
- Late, incomplete, copied or unverifiable submissions may be penalized as per faculty and departmental guidelines.

b. Course Outcomes

CO	Course Outcome	Bloom's Level
CO1	Explain cybersecurity principles, security goals and threat landscapes.	L2 - Understand
CO2	Apply system hardening and vulnerability assessment techniques.	L3 - Apply
CO3	Implement authentication, encryption and secure communication mechanisms.	L3 - Apply



CO	Course Outcome	Bloom's Level
CO4	Analyze common cyberattacks and related defensive controls.	L4 - Analyze
CO5	Apply incident response and log analysis practices.	L3 - Apply

4. Lab Experiments - CO Mapping Table

S. No.	Experiment / Assessment Title	Mapped CO(s)	Bloom's Level	Marks
1	Lab 1: Threat Mapping and Breach Analysis Workbench	CO1	L4 - Analyze	100
2	Lab 2: Hardening a Personal Linux System	CO2	L3 - Apply	100
3	Lab 3: Secure Communication & Cryptography Explorer	CO3	L3 - Apply	100
4	Lab 4: Vulnerability Testing with OWASP Tools	CO4, CO5	L4 - Analyze	100
5	Capstone: Small Business Security Defence Planner	CO1, CO2, CO3, CO4, CO5	L5 - Evaluate/Create	200
6	Viva 1	CO1, CO2	L3/L4 - Apply/Analyze	100
7	Viva 2	CO3, CO4, CO5	L4/L5 - Analyze/Evaluate	100
8	Final Lab File Upload	CO1, CO2, CO3, CO4, CO5	-	50
	Total Marks			850



Lab 1: Threat Mapping and Breach Analysis

Mapped COs: CO1, CO4, CO5

Bloom: L4 - Analyze

Total: 100 marks

Problem Statement	Analyze a real or publicly documented cybersecurity breach and map the attack to security goals, threat categories and cyber kill chain stages. Classify the threat, identify affected assets, estimate risk and prepare a concise, mitigation-oriented analysis report.
Lab Instructions	Use MITRE ATT&CK, NIST CSF and public breach reports as guided by faculty. Show threat classification, kill chain mapping, a risk matrix and mitigation strategies in a structured report.
Tools	MITRE ATT&CK Navigator, NIST CSF reference, public breach reports and CISA advisories, and a documentation/spreadsheet tool.

Running example used throughout this lab: a fictional but realistic ransomware-and-data-theft breach at a mid-sized company, “Acme HealthCare”, whose patient database was accessed after an employee opened a phishing email. Each question applies its algorithm to this same case so you can see the full analysis come together.



Q1

Classify the main threats using the CIA triad and security goals

5 Marks

Objective: Identify which of Confidentiality, Integrity and Availability were violated in the breach and state the security goal affected by each attacker action.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	Read the breach summary and list every attacker action you can identify.	Breach report	List of actions
2	ANALYSE	For each action, decide whether it broke Confidentiality, Integrity or Availability.	CIA triad	C / I / A tag per action
3	ANALYSE	State the specific security goal that failed (e.g. "data secrecy", "data accuracy", "service uptime").	Security goals	Goal named per action
4	REPORT	Summarise which CIA element was most severely impacted overall.	—	One-line verdict
5	REPORT	Record the classification in a small table for the report.	Table template	Completed CIA table

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

Applying the steps to Acme HealthCare

The attacker (1) stole 50,000 patient records, (2) encrypted the database with ransomware, and (3) took the booking portal offline. Each action is tagged against the CIA triad:

Attacker action	CIA element broken	Security goal failed
Stole patient records	Confidentiality	Data secrecy
Encrypted the database	Integrity	Data accuracy / access
Booking portal taken offline	Availability	Service uptime

Overall verdict: Confidentiality was most severely impacted, because sensitive medical data left the organisation permanently.

Reusable Template

CIA Classification of E-Commerce Data Breach



Attacker action	CIA element (C / I / A)	Security goal failed
Stole customer credit - card details	CONFIDENTIALITY	Data privacy
Changed product price in database	INTEGRITY	Data accuracy
Took the shopping website offline	AVAILABILITY	Service uptime

Your Output / Screenshot (paste result here):

overall verdict: CONFIDENTIALITY was most severely impact because sensitive customers payment information was exposed.



Q2

Collect breach information from credible sources and summarise it

5 Marks

Objective: Gather verifiable facts about the breach from reputable public sources and write a short, neutral incident background.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	Identify 2–3 credible sources (vendor advisory, CISA, established news).	CISA / news / vendor blog	Source list
2	ANALYSE	Extract the key facts: who, what, when, how discovered, what data.	5W notes	Fact sheet
3	TEST	Cross-check each fact against a second source to avoid rumour.	Second source	Facts confirmed
4	ANALYSE	Note anything still unknown or disputed as an open question.	—	Open-questions list
5	REPORT	Write a 4–6 sentence neutral summary with citations.	Summary template	Incident background

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

Sample incident background (Acme HealthCare)

On 3 March, Acme HealthCare disclosed that attackers accessed its patient database after an employee opened a phishing email on 24 February. Roughly 50,000 records were exfiltrated and systems were encrypted with ransomware. The intrusion was detected when staff could not log in to the booking portal. The company engaged an incident-response firm and notified the data-protection regulator. (Sources: company statement; national CERT advisory.)

Fact	Detail
Who	Acme HealthCare (healthcare provider)
What	Phishing email leading to ransomware and data theft
When	Initial access 24 Feb; publicly disclosed 3 Mar
How discovered	Staff unable to log in to the booking portal
Data affected	~50,000 patient records
Sources	Company statement; national CERT advisory



Reusable Template

XYZ Bank was targeted by a phishing attack on 15 May. The attackers used phishing to infect systems with malware and steal customer information. The attack was discovered on 18 May after the security team noticed unusual login activity. Approximately 25,000 customer records were affected.

Fact	Detail
Who	XYZ Bank (financial institution)
What	Phishing attack leading to malware infection and customer data theft
When	Initial attack on 15 may; incident discovered on 18 may
How discovered	Security team detected unusual login activity
Data affected	Approximately 25,000 customer records
Sources	Company incident report; national CERT advisory

Your Output / Screenshot (paste result here):



Q3

Map the breach to Cyber Kill Chain stages / MITRE ATT&CK

5 Marks

Objective: Place each attacker action into the correct Cyber Kill Chain stage and, where possible, tag it with a MITRE ATT&CK technique ID.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	List the seven Cyber Kill Chain stages as your framework.	Cyber Kill Chain	Stage list ready
2	ANALYSE	Assign each attacker action to the stage where it belongs.	Kill chain mapping	Action per stage
3	ANALYSE	Look up a matching ATT&CK technique for each action.	ATT&CK Navigator	Technique IDs (T####)
4	TEST	Mark the earliest stage where detection was realistically possible.	—	Detection point flagged
5	REPORT	Record the mapping as a table for the report.	Table template	Kill chain table

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

Kill chain and ATT&CK mapping (Acme HealthCare)

Kill Chain stage	Observed action	ATT&CK technique
Reconnaissance	Employee emails researched	T1591
Delivery	Phishing email sent	T1566
Exploitation	Malicious attachment opened	T1204
Installation	Ransomware dropped	T1486
Actions on Objectives	Data exfiltrated	T1041

Earliest realistic detection: the Delivery stage. An email gateway scanning attachments could have quarantined the phishing message before it ever reached the user.

Reusable Template

Kill chain mapping : Spear-Phishing Attack



Kill Chain stage	Observed action	ATT&CK technique (optional)
Reconnaissance	Company website and employees profiles researched	T1593
Delivery	Spear-phishing email sent	T1566
Exploitation	Malicious link clicked	T1204
Installation	Malware installed on the computer	T1204
Command &Control	Malware connected to attacker server	T1071
Actions on Objectives	Sensitive files collected and transferred	T1041

Earliest realization: occurred on 18 May, when the security team detected unusual login activity, indicating that the spear-phishing attack had potentially compromised systems and exposed customer data.



Q4

Create a risk matrix (asset, threat, vulnerability, likelihood, impact, risk)

5 Marks

Objective: Build a simple risk matrix that rates each asset-threat pair by likelihood and impact and derives an overall risk level.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	List the key assets involved in the breach.	Asset inventory	Asset list
2	ANALYSE	For each asset, name the threat and the vulnerability it exploited.	Threat/vuln notes	Threat + vuln per asset
3	TEST	Rate likelihood as Low / Medium / High.	Likelihood scale	Likelihood set
4	TEST	Rate impact as Low / Medium / High.	Impact scale	Impact set
5	ANALYSE	Combine likelihood $\times$ impact to get the risk level and colour it.	Risk = $L \times I$	Colour-coded risk
6	REPORT	Sort the matrix so the highest risks appear first.	Sort by risk	Prioritised matrix

Phase key: • SETUP • CONFIGURE • EXECUTE • TEST • ANALYSE • REPORT • FINISH

Worked Example (with sample values & output)

Risk matrix (Acme HealthCare)

Asset	Threat	Vulnerability	Likelihood	Impact	Risk
Patient DB	Data theft	Weak email filter	High	High	CRITICAL
Booking portal	Ransomware	Unpatched server	Medium	High	HIGH
Staff accounts	Credential theft	No MFA	High	Medium	HIGH

Reading the matrix: the Patient DB is the top priority (High likelihood and High impact = Critical), so its mitigations are addressed first in Q5.

Reusable Template

Risk matrix template (Acme HealthCare: Watering Hole Attack)



Asset	Threat	Vulnerability	Likelihood	Impact	Risk
Staff workstations	Malware via compromised industry website	Outdated browser and plugins	High	High	CRITICAL
Patient DB	Data theft after drive-by infection	Flat network, no segmentation	Medium	High	HIGH
Staff accounts	Credential theft from fake login redirect	No MFA	High	Medium	HIGH
Web proxy / DNS	Users reach malicious domain	No web filtering or DNS blocklist	Medium	Medium	MEDIUM

Your Output / Screenshot (paste result here):

the staff workstations are the top priority (High likelihood and High impact = Critical). Staff routinely visit trusted industry sites, and one compromised site can infect many machines at once, so mitigations (patching, browser isolation, web filtering) are addressed first in Q5.



Q5

Identify preventive, detective and corrective mitigations

5 Marks

Objective: For the top risks, propose controls in three categories: preventive (stop it), detective (spot it) and corrective (recover from it).

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	Take the highest-risk items from the Q4 matrix.	Q4 risk matrix	Top risks selected
2	ANALYSE	For each, list a preventive control that would stop the attack.	Control catalogue	Preventive control
3	ANALYSE	Add a detective control that would raise an early alert.	Control catalogue	Detective control
4	ANALYSE	Add a corrective control that limits or reverses damage.	Control catalogue	Corrective control
5	TEST	Check each control is realistic and maps back to a risk.	—	Controls validated
6	REPORT	Record the three-column control table.	Table template	Mitigation table

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

Mitigation controls (Acme HealthCare)

Risk	Preventive (stop it)	Detective (spot it)	Corrective (recover)
Data theft	Email filtering + MFA	DLP alerts on large exports	Breach notification + credit monitoring
Ransomware	Patch servers + app allowlist	EDR behaviour alerts	Offline backups + restore plan

Each row covers all three control types, so even if prevention fails, the attack can still be detected and recovered from.

Reusable Template

Three-lines-of-defence template (copy and fill)

Risk	Preventive	Detective	Corrective



Risk	Preventive	Detective	Corrective

Your Output / Screenshot (paste result here):

Q6	Compare against NIST CSF and identify control gaps	5 Marks
-----------	---	----------------

Objective: Assess the victim organisation against the five NIST CSF functions and list where controls were missing or weak.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	Write down the five NIST CSF functions as a checklist.	NIST CSF	Identify/Protect/Detect/Respond/Recover
2	ANALYSE	For each function, ask what the organisation did or failed to do.	Breach facts	Notes per function
3	TEST	Rate each function as Adequate, Partial or Missing.	Maturity scale	Rating per function
4	ANALYSE	For weak functions, state the specific control gap.	—	Gap statement
5	REPORT	Record the CSF assessment table and highlight the biggest gap.	Table template	CSF gap table

Phase key: • SETUP • CONFIGURE • EXECUTE • TEST • ANALYSE • REPORT • FINISH

Worked Example (with sample values & output)

NIST CSF gap assessment (Acme HealthCare)

CSF function	Rating	Gap identified
Identify	Partial	No up-to-date asset / data inventory



Protect	Missing	No MFA; weak email filtering
Detect	Missing	No monitoring of large data exports
Respond	Partial	IR plan existed but response was slow
Recover	Partial	Backups existed but were not tested

Biggest gap: the Detect function. The exfiltration went unnoticed until systems were locked, showing monitoring was the weakest area.

Reusable Template

CSF assessment template (rate each Adequate / Partial / Missing)

CSF function	Rating	Gap identified
Identify	Partial	Asset inventory exists but several cloud systems are not recorded
Protect	Partial	password policy exists, but MFA is not enabled for all users.
Detect	Missing	No continuous monitoring or alerting for suspicious login activity
Respond	partial	Incident response procedure exists, but roles and responsibilities are unclear
Recover	Adequate	Regular backup are maintained and recovery procedures are tested.

Your Output / Screenshot (paste result here):

Detect is rated **Missing** with the gap "No monitoring of large data exports." The note below the table confirms it: the exfiltration went unnoticed until systems were locked, which shows monitoring was the weakest area.



Q7

Prepare and present the breach analysis report

10 Marks

Objective: Combine the outputs of Q1–Q6 into a short, structured breach analysis report with findings, risk interpretation and lessons learned.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	Collect your completed tables from Q1–Q6.	Q1–Q6 outputs	All tables gathered
2	REPORT	Write an executive summary (3–4 sentences) of the breach.	Summary	Executive summary
3	REPORT	Add sections: background, CIA, kill chain, risk matrix, mitigations, CSF gaps.	Report structure	Draft report
4	ANALYSE	Write a lessons-learned section with 3–5 concrete takeaways.	—	Lessons learned
5	TEST	Proofread for clarity and check every claim traces to evidence.	Review checklist	Report reviewed
6	REPORT	Export the report and prepare a short presentation.	PDF / slides	Final deliverables

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

Report structure (Acme HealthCare)

Section	Content	Comes from
1. Executive Summary	3-4 sentence overview of the breach	New
2. Incident Background	Verified facts and timeline	Q2
3. Threat Classification	CIA impact of each action	Q1
4. Attack Mapping	Kill chain + ATT&CK techniques	Q3
5. Risk Assessment	Prioritised risk matrix	Q4
6. Recommended Mitigations	Preventive / detective / corrective	Q5
7. NIST CSF Gap Analysis	Function ratings and gaps	Q6



8. Lessons Learned

3-5 concrete, actionable takeaways

New

Sample lessons learned: deploy MFA on all accounts; add attachment sandboxing to email; monitor for large data exports; and test backups quarterly.

Reusable Template

Submission checklist

✓	Check before submitting
☐	Executive summary present
☐	All six analysis tables (Q1-Q6) included
☐	Every claim backed by a source or table
☐	Lessons learned are specific and actionable
☐	Report + GitHub link uploaded to ZOSIMA

Your Output / Screenshot (paste result here):

So	Section	Content	Comes From
1.	Executive Summary	Brief overview of Security Incident.	New
2.	Incident Background	Verified facts, affected systems and timeline	Q1
3.	Threat Classification	Type of threat and impact on CIA	Q2
4.	Attack Analysis	Attack vector, techniques and method used	Q3
5.	Risk Assessment	Likelihood, impact and priority of risk	Q4
6.	Recommended Mitigations	Preventive, detective and corrective controls	Q5
7.	NIST CSF Analysis	Identify, Protect, Detect, Respond and Recover gaps	Q6
8.	Lessons Learned	3-5 Key lessons and future improvements	New

Sample lessons learned:

The incident showed that the biggest weakness was the ability to detect an attack in progress, followed by weak access controls. Acme HealthCare's tested backups were the one strong point, and they shortened



recovery. Priority actions for the next quarter are to introduce continuous monitoring, enforce MFA for all users, and clarify incident response roles. Together these close the main gaps found in the NIST CSF analysis.



Lab 1 Assessment Scheme

Assessment	Basis	Marks
Question 1	Identify and classify the main threats using the CIA triad and basic security goals.	5
Question 2	Collect relevant breach information from credible public sources and summarise the incident background.	5
Question 3	Map the breach activities to cyber kill chain stages or MITRE ATT&CK tactics.	5
Question 4	Create a simple risk matrix (asset, threat, vulnerability, likelihood, impact, risk level).	5
Question 5	Identify preventive, detective and corrective mitigation strategies.	5
Question 6	Compare the breach against NIST CSF and identify control gaps.	5
Question 7	Prepare and present a short breach analysis report with findings and lessons learned.	10
Lab Assessment on ZOSIMA	Live four-criteria rubric (efforts & execution, output & correctness, testing & verification, explanation & authenticity), verified via GitHub.	40
Total		100



Lab 2: Hardening a Personal Linux System

Mapped COs: CO2	Bloom: L3 - Apply	Total: 100 marks
Problem Statement	Configure and harden a Linux virtual machine by applying basic security controls related to patching, user access, file permissions, firewall configuration and service exposure. Document each change and explain its security impact.	
Lab Instructions	Use an Ubuntu/Linux VM, VirtualBox or VMware, UFW/iptables, nmap, systemctl and chmod/chown. Show before-and-after states of each hardening step with screenshots and explanations.	
Tools	Ubuntu/Linux VM, VirtualBox/VMware, apt, useradd/groupadd, chmod/chown, ufw or iptables, systemctl and nmap.	

Running example used throughout this lab: a fresh Ubuntu Server virtual machine, “lab-vm”, that begins unpatched with a default user and an open web service. Each question hardens one aspect of this same VM, so by the end you have a fully secured host with before-and-after evidence.



Q1

Install / configure the VM and apply the latest security patches

5 Marks

Objective: Bring a Linux virtual machine up to date so that known vulnerabilities in installed packages are closed before any other hardening.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	Take a clean snapshot of the VM so you can roll back if needed.	VirtualBox / VMware	Snapshot saved
2	CONFIGURE	Refresh the package index to learn what updates are available.	<code>sudo apt update</code>	Package lists updated
3	CONFIGURE	Upgrade all installed packages to apply security patches.	<code>sudo apt upgrade -y</code>	Packages upgraded
4	TEST	Confirm nothing is left upgradable.	<code>apt list --upgradable</code>	List is empty
5	REPORT	Record the kernel/OS version as evidence of patch level.	<code>uname -r ; lsb_release -a</code>	Version noted

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

Applying the steps to lab-vm

Running the update and upgrade on the fresh VM shows several security updates being applied. Afterwards the upgradable list is empty, confirming the system is patched.

Command	Purpose	Expected result
<code>sudo apt update</code>	Refresh package index	Lists refreshed from mirrors
<code>sudo apt upgrade -y</code>	Install available updates	Security patches applied
<code>apt list --upgradable</code>	Verify none remain	(empty) after upgrade

Sample output:

```
$ apt list --upgradable
Listing... Done
$ (no packages listed – system is up to date)
```

Reusable Template

Patch-evidence template



Item	Value
OS version	
Kernel version	
Date patched	
Updates applied	

Your Output / Screenshot (paste result here):



Q2

Create users and groups and verify least-privilege access

5 Marks

Objective: Set up user accounts and groups so each user has only the access they need, following the principle of least privilege.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	CONFIGURE	Create a dedicated group for a team or role.	<code>sudo groupadd secteam</code>	Group in /etc/group
2	CONFIGURE	Create a user with a home directory and shell.	<code>sudo useradd -m -s /bin/bash analyst</code>	User in /etc/passwd
3	CONFIGURE	Add the user to the appropriate group.	<code>sudo usermod -aG secteam analyst</code>	Membership set
4	TEST	Check the user's group membership.	<code>groups analyst</code>	Groups listed
5	TEST	Confirm the user has no unintended sudo rights.	<code>sudo -l -U analyst</code>	Only intended rights

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

Applying the steps to lab-vm

A security group and an analyst account are created, then the account is checked to confirm it only belongs to the intended groups and has no administrative privileges.

User	Primary group	Extra groups	Sudo rights
analyst	analyst	secteam	None
admin	admin	sudo	Full

Sample output:

```
$ groups analyst
analyst : analyst secteam
```

Reusable Template

User-access record template



User	Groups	Least-privilege verified? (Y/N)

Your Output / Screenshot (paste result here):



Q3

Configure file and directory permissions (chmod / chown)

5 Marks

Objective: Restrict access to sensitive files using ownership and permission bits, and verify the change.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	Identify a sensitive file that needs restricted access.	ls -l sensitive.conf	Current permissions
2	CONFIGURE	Set the owner and group appropriately.	sudo chown root:secteam sensitive.conf	Ownership changed
3	CONFIGURE	Apply restrictive permission bits.	sudo chmod 640 sensitive.conf	Mode = -rw-r----
4	TEST	Re-check the permissions to confirm.	ls -l sensitive.conf	New permissions shown
5	TEST	Test that an unauthorised user is denied.	sudo -u nobody cat sensitive.conf	Permission denied

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

Applying the steps to lab-vm

The configuration file starts world-readable. After changing ownership and applying mode 640, only root and the secteam group can read it, and other users are denied.

Stage	Owner:Group	Permissions	Who can read
Before	user:user	-rw-r--r--	Everyone
After	root:secteam	-rw-r----	root + secteam only

Sample output:

```
$ ls -l sensitive.conf
-rw-r----- 1 root secteam 220 Mar 5 sensitive.conf
```

Reusable Template

Permission-change record template

File	Before (owner / mode)	After (owner / mode)



File	Before (owner / mode)	After (owner / mode)

Your Output / Screenshot (paste result here):



Q4

Enable and test firewall rules (UFW / iptables)

5 Marks

Objective: Configure a host firewall with a default-deny inbound policy and allow only the services you need.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	CONFIGURE	Set the default policy to deny inbound traffic.	<code>sudo ufw default deny incoming</code>	Default deny set
2	CONFIGURE	Allow outbound traffic by default.	<code>sudo ufw default allow outgoing</code>	Default allow set
3	CONFIGURE	Allow only required inbound services (e.g. SSH).	<code>sudo ufw allow 22/tcp</code>	Rule added
4	EXECUTE	Enable the firewall.	<code>sudo ufw enable</code>	Firewall active
5	TEST	Verify the rule set.	<code>sudo ufw status verbose</code>	Rules listed

Phase key: ● **SETUP** · ● **CONFIGURE** · ● **EXECUTE** · ● **TEST** · ● **ANALYSE** · ● **REPORT** · ● **FINISH**

Worked Example (with sample values & output)

Applying the steps to lab-vm

After setting a default-deny inbound policy and allowing only SSH, the firewall status shows a small, deliberate rule set instead of everything being open.

Direction	Default policy	Explicit allow	Result
Inbound	Deny	22/tcp (SSH)	Only SSH reachable
Outbound	Allow	-	Normal traffic works

Sample output:

```
$ sudo ufw status verbose
Status: active
Default: deny (incoming), allow (outgoing)
22/tcp          ALLOW IN    Anywhere
```

Reusable Template

Firewall rule record template



Port / service	Action	Reason it is needed
22/tcp (SSH)	Allow	

Your Output / Screenshot (paste result here):



Q5

Identify open ports/services with nmap and disable the unnecessary ones

5 Marks

Objective: Discover which ports are actually open, then reduce the attack surface by disabling services you do not need.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	TEST	Scan the host for open TCP ports.	<code>nmap -sT -p-127.0.0.1</code>	Open-port list
2	ANALYSE	Match each open port to a running service.	<code>ss -tuln</code>	Port-to-service map
3	ANALYSE	Decide which services are unnecessary.	—	Disable list
4	CONFIGURE	Stop and disable each unneeded service.	<code>sudo systemctl disable --now <svc></code>	Service inactive
5	TEST	Re-scan to confirm the port is now closed.	<code>nmap -sT -p-127.0.0.1</code>	Fewer open ports

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

Applying the steps to lab-vm

The first scan finds an unused Apache service on port 80. After disabling it, the re-scan shows only SSH remaining open, shrinking the attack surface.

Port	Service	Needed?	Action taken
22/tcp	SSH	Yes	Keep
80/tcp	Apache (unused)	No	Disable service

Sample output:

Before: 22/tcp open 80/tcp open
After : 22/tcp open

Reusable Template

Open-ports record template



Port	Service	Needed? (Y/N)	Action

Your Output / Screenshot (paste result here):



Q6

Verify and document system hardening (audit / checklist)

5 Marks

Objective: Confirm the hardening actually took effect using audit commands, and record it against a checklist.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	TEST	Confirm the system is patched.	apt list --upgradable	Empty list
2	TEST	Confirm least-privilege users are in place.	cat /etc/passwd ; groups	Expected users only
3	TEST	Confirm the firewall is active with the right rules.	sudo ufw status verbose	Active + minimal rules
4	TEST	Confirm only required ports are open.	nmap -sT -p-127.0.0.1	Minimal ports
5	REPORT	Tick each item on the hardening checklist.	Checklist	Completed checklist

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

Hardening checklist (lab-vm)

Control	Verified with	Status
Patches applied	apt list --upgradable	PASS
Least-privilege users	groups / sudo -l	PASS
File permissions	ls -l	PASS
Firewall active	ufw status	PASS
Minimal open ports	nmap	PASS

Every control shows PASS, so the VM meets the baseline hardening standard for this lab.

Reusable Template

Hardening checklist template

Control	Verified with	Status (PASS/FAIL)
Patches applied		



Control	Verified with	Status (PASS/FAIL)
Least-privilege users		
File permissions		
Firewall active		
Minimal open ports		

Your Output / Screenshot (paste result here):



Q7

Prepare a hardening checklist and explain each control's impact

10 Marks

Objective: Produce a clear report that lists each hardening control, its before-and-after state and how it reduces exposure, with supporting screenshots.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	Gather your evidence and outputs from Q1-Q6.	Q1-Q6 outputs	Evidence collected
2	REPORT	For each control, write what changed and why it matters.	Report	Control explanations
3	REPORT	Attach before-and-after screenshots for each control.	Screenshots	Evidence attached
4	ANALYSE	Summarise how the attack surface was reduced overall.	—	Impact summary
5	REPORT	Review and export the report, then push evidence to GitHub.	PDF + GitHub	Final deliverable

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

Report structure (lab-vm)

Control	Before	After	Security impact
Patching	Outdated packages	Fully patched	Closes known CVEs
Least privilege	Shared admin user	Role-based users	Limits blast radius
Firewall	All ports open	Default-deny + SSH	Blocks unwanted access
Services	Unused Apache on	Apache disabled	Smaller attack surface

Overall: the VM moved from an exposed default state to a hardened baseline, with each control mapped to the specific risk it reduces.

Reusable Template

Report submission checklist



✓	Check before submitting
☐	All six controls documented with before/after
☐	Screenshots attached for each control
☐	Security impact explained per control
☐	Report + GitHub link uploaded to ZOSIMA

Your Output / Screenshot (paste result here):



Lab 2 Assessment Scheme

Assessment	Basis	Marks
Question 1	Install/configure a Linux VM and update it with the latest security patches.	5
Question 2	Create users and groups, assign roles and verify least-privilege access.	5
Question 3	Configure file and directory permissions using chmod, chown and related commands.	5
Question 4	Enable and test UFW or iptables firewall rules for selected traffic.	5
Question 5	Identify open ports/services with nmap and disable unnecessary services.	5
Question 6	Verify and document system hardening using audit commands or a checklist.	5
Question 7	Prepare a hardening checklist explaining how each control reduces exposure.	10
Lab Assessment on ZOSIMA	Live four-criteria rubric (efforts & execution, output & correctness, testing & verification, explanation & authenticity), verified via GitHub.	40
Total		100



Lab 3: Secure Communication and Encryption Basics

Mapped COs: CO3

Bloom: L3 - Apply

Total: 100 marks

Problem Statement	Demonstrate basic secure communication mechanisms by using encryption, hashing, digital certificates and packet inspection. Understand how confidentiality, integrity and authenticity are implemented in common workflows.
Lab Instructions	Use OpenSSL, Wireshark, the browser certificate viewer and a local test-file environment. Clearly show encryption outputs, hash values, certificate details and packet-capture observations.
Tools	OpenSSL, sha256sum, a web browser certificate viewer, Wireshark and a local test file.

Running example used throughout this lab: a single confidential file, “secret.txt”, containing a short patient note. Across the questions you protect this same file with symmetric and public-key encryption, verify its integrity with hashing, and observe how it travels over HTTP versus HTTPS.



Q1

Encrypt and decrypt a file with a symmetric key (OpenSSL)

5 Marks

Objective: Use symmetric AES encryption to protect a file, then decrypt it and confirm the content is unchanged.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	Create a sample plaintext file.	echo "secret note" > secret.txt	File created
2	CONFIGURE	Generate a random symmetric key.	openssl rand -hex 32 > key.hex	Key file created
3	EXECUTE	Encrypt the file with AES-256.	openssl enc -aes-256-cbc ...	secret.enc (unreadable)
4	EXECUTE	Decrypt the file back.	openssl enc -d -aes-256-cbc ...	decrypted.txt
5	TEST	Compare the decrypted file with the original.	diff secret.txt decrypted.txt	No differences

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

Applying the steps to secret.txt

Stage	File	Readable?
Original	secret.txt	Yes
Encrypted	secret.enc	No
Decrypted	decrypted.txt	Yes

Sample output:

```
$ diff secret.txt decrypted.txt
$ (no output – files are identical)
```

Reusable Template

Commands to use

```
openssl rand -hex 32 > key.hex
openssl enc -aes-256-cbc -salt -in secret.txt -out secret.enc -pass file:./key.hex
openssl enc -d -aes-256-cbc -in secret.enc -out decrypted.txt -pass file:./key.hex
```



Your Output / Screenshot (paste result here):



Q2

Protect a message with a public-private key workflow

5 Marks

Objective: Use an RSA key pair to encrypt a message with the public key and decrypt it with the private key.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	CONFIGURE	Generate an RSA private key.	<code>openssl genrsa -out private.pem 2048</code>	private.pem created
2	CONFIGURE	Extract the matching public key.	<code>openssl rsa -in private.pem -pubout -out public.pem</code>	public.pem created
3	EXECUTE	Encrypt a message using the public key.	<code>openssl pkeyutl -encrypt -pubin ...</code>	msg.enc created
4	EXECUTE	Decrypt the message using the private key.	<code>openssl pkeyutl -decrypt -inkey private.pem ...</code>	Message recovered
5	TEST	Confirm the recovered message matches the original.	<code>diff msg.txt msg.dec</code>	No differences

Phase key: • **SETUP** • **CONFIGURE** • **EXECUTE** • **TEST** • **ANALYSE** • **REPORT** • **FINISH**

Worked Example (with sample values & output)

Applying the steps to a short message

Key used	Operation	Who can do it
Public key	Encrypt	Anyone (it is public)
Private key	Decrypt	Only the owner

This shows why public-key crypto solves key distribution: the sender needs only the freely shared public key, while only the private-key holder can read the message.

Reusable Template

Commands to use

```
openssl genrsa -out private.pem 2048
openssl rsa -in private.pem -pubout -out public.pem
openssl pkeyutl -encrypt -pubin -inkey public.pem -in msg.txt -out msg.enc
openssl pkeyutl -decrypt -inkey private.pem -in msg.enc -out msg.dec
```



Your Output / Screenshot (paste result here):



Q3

Verify file integrity with SHA-256 hashing

5 Marks

Objective: Show that a cryptographic hash detects any change to a file, however small.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	EXECUTE	Compute the SHA-256 hash of the original file.	sha256sum secret.txt	64-char digest
2	REPORT	Record the digest as the baseline.	—	Baseline hash saved
3	CONFIGURE	Make a small change to the file.	echo x >> secret.txt	File modified
4	EXECUTE	Re-compute the hash.	sha256sum secret.txt	New digest
5	TEST	Compare the two digests to detect tampering.	compare	Digests differ

Phase key: • SETUP • CONFIGURE • EXECUTE • TEST • ANALYSE • REPORT • FINISH

Worked Example (with sample values & output)

Applying the steps to secret.txt

State	SHA-256 (first 16 chars)	Match baseline?
Original	9f2c4a1b7e0d8c33...	Baseline
After one byte added	3a77e15c90bb2f48...	No - changed

Even adding a single byte produces a completely different digest, which is exactly why hashing is used for integrity checks.

Reusable Template

Commands to use

```
sha256sum secret.txt      # record this baseline
echo 'x' >> secret.txt    # simulate a change
sha256sum secret.txt      # compare with baseline
```

Your Output / Screenshot (paste result here):





Q4

Analyse a digital certificate in the browser

5 Marks

Objective: Inspect a real HTTPS certificate and read its issuer, subject, validity and chain of trust.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	Open a known HTTPS website in the browser.	https://example.com	Padlock shown
2	EXECUTE	Open the certificate viewer from the padlock.	Browser cert viewer	Certificate panel
3	ANALYSE	Read the subject and issuer fields.	—	Names identified
4	ANALYSE	Note the validity (not-before / not-after) dates.	—	Validity period
5	ANALYSE	Follow the chain up to the trusted root CA.	—	Chain to root

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

Certificate details (example.com)

Field	Value (example)
Subject	example.com
Issuer	DigiCert Global G2 CA
Valid from	01 Jan 2026
Valid to	01 Apr 2026
Chain	example.com -> Intermediate CA -> Trusted Root

The chain links the site's certificate through an intermediate CA to a root the browser already trusts, which is what turns the padlock green.

Reusable Template

Command-line alternative (optional)

```
openssl s_client -connect example.com:443 -servername example.com </dev/null \
```



| openssl x509 -noout -issuer -subject -dates

Your Output / Screenshot (paste result here):



Q5

Compare HTTP vs HTTPS traffic in Wireshark

5 Marks

Objective: Capture traffic to an HTTP site and an HTTPS site and show that only the HTTP content is readable.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	Start a Wireshark capture on the active interface.	Wireshark	Capture running
2	EXECUTE	Browse to an HTTP (non-secure) page.	http://...	HTTP packets
3	EXECUTE	Browse to an HTTPS (secure) page.	https://...	TLS packets
4	ANALYSE	Filter and inspect the HTTP request contents.	http filter	Readable text
5	ANALYSE	Inspect the HTTPS/TLS stream contents.	tls filter	Encrypted bytes

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

Capture comparison

Site type	Protocol	Payload visible?	What you see
HTTP	http	Yes	Readable URLs and text
HTTPS	TLS	No	Encrypted application data

The HTTP capture exposes the request in plain text, while the HTTPS capture shows only encrypted bytes, demonstrating the value of TLS.

Reusable Template

Useful Wireshark display filters

```
http          # show only HTTP
tls           # show only TLS/HTTPS
ip.addr == <site-ip> # focus on one server
```

Your Output / Screenshot (paste result here):





Q6

Set up and observe an SSL/TLS (or VPN) encrypted connection

5 Marks

Objective: Create a simple TLS-protected connection and confirm with a capture that the traffic is encrypted.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	CONFIGURE	Generate a self-signed certificate for a local server.	<code>openssl req -x509 -newkey rsa:2048 ...</code>	cert.pem + key.pem
2	EXECUTE	Start a local TLS server using the certificate.	<code>openssl s_server -accept 4443 ...</code>	Server listening
3	EXECUTE	Connect a TLS client to the server.	<code>openssl s_client -connect localhost:4443</code>	TLS handshake OK
4	ANALYSE	Capture the connection in Wireshark.	Wireshark (port 4443)	TLS records
5	TEST	Confirm the application data is encrypted.	tls filter	No cleartext

Phase key: • SETUP • CONFIGURE • EXECUTE • TEST • ANALYSE • REPORT • FINISH

Worked Example (with sample values & output)

TLS connection observation

Phase	What happens	Visible in capture
Handshake	Client & server agree keys	Cert + key exchange
Data transfer	Message sent over channel	Encrypted only

The handshake is visible (that is normal), but the actual message content is encrypted, so an eavesdropper cannot read it.

Reusable Template

Commands to use

```
openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 30 -nodes
openssl s_server -accept 4443 -cert cert.pem -key key.pem
openssl s_client -connect localhost:4443
```

Your Output / Screenshot (paste result here):



K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION



Q7

Report how encryption, hashing and certificates secure communication

10 Marks

Objective: Bring together the evidence from Q1-Q6 into a short report explaining how each mechanism provides confidentiality, integrity or authenticity.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	Collect your outputs from Q1-Q6.	Q1-Q6 outputs	Evidence gathered
2	ANALYSE	Map each mechanism to the security property it provides.	—	Mapping table
3	REPORT	Explain each with your own captured evidence.	Report	Explanations written
4	REPORT	Attach screenshots of encryption, hashes and captures.	Screenshots	Evidence attached
5	REPORT	Review, export and push the report to GitHub.	PDF + GitHub	Final deliverable

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

Mechanism-to-property mapping

Mechanism	Security property	Evidence from this lab
AES / RSA encryption	Confidentiality	Q1, Q2 encrypted files
SHA-256 hashing	Integrity	Q3 changed digest
Digital certificate / TLS	Authenticity	Q4, Q5, Q6 captures

The report should tie each row back to the actual output you produced, showing you understand which property each mechanism protects.

Reusable Template

Report submission checklist

✓	Check before submitting
☐	Each mechanism mapped to its security property



✓	Check before submitting
☐	Screenshots of encryption, hash and capture attached
☐	Plain-language explanation for each
☐	Report + GitHub link uploaded to ZOSIMA

Your Output / Screenshot (paste result here):



Lab 3 Assessment Scheme

Assessment	Basis	Marks
Question 1	Use OpenSSL to generate symmetric keys and encrypt/decrypt a sample file.	5
Question 2	Simulate a public-private key workflow to protect a message or file.	5
Question 3	Generate a SHA-256 hash and verify integrity after modifying the file.	5
Question 4	Analyse a digital certificate (issuer, subject, validity, chain).	5
Question 5	Use Wireshark to compare HTTP and HTTPS traffic.	5
Question 6	Configure and test a TLS/VPN connection and observe encrypted traffic.	5
Question 7	Report how encryption, hashing and certificates support secure communication.	10
Lab Assessment on ZOSIMA	Live four-criteria rubric (efforts & execution, output & correctness, testing & verification, explanation & authenticity), verified via GitHub.	40
Total		100



Lab 4: Vulnerability Testing with OWASP Tools

Mapped COs: CO2, CO4, CO5		Bloom: L4 / L5	Total: 100 marks
Problem Statement	Set up a deliberately vulnerable web application in a safe lab environment and perform basic web vulnerability testing. Simulate selected attacks, observe their impact, analyse logs and recommend remediation.		
Lab Instructions	Use DVWA, OWASP ZAP or Burp Suite Community, a local web-server stack, browser developer tools and log files. All testing must be performed only against the local, intentionally vulnerable target.		
Tools	DVWA on a local Apache/PHP/MySQL stack, OWASP ZAP or Burp Suite Community, browser developer tools and server/application logs.		

Running example used throughout this lab: a locally hosted DVWA instance. Each question exercises one part of a realistic web assessment against this same safe target: standing it up, testing SQL injection and XSS, proxying traffic, reading logs, and applying and verifying a fix.



Q1

Set up DVWA (or equivalent) in an isolated lab environment

5 Marks

Objective: Deploy an intentionally vulnerable web application safely on an isolated local stack.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	Install the local web stack (Apache, PHP, MySQL).	LAMP / XAMPP	Stack running
2	SETUP	Deploy the DVWA files into the web root.	DVWA	Site loads
3	CONFIGURE	Create and configure the DVWA database.	Setup page	DB connected
4	TEST	Confirm the app is only reachable locally.	127.0.0.1 only	Not exposed externally
5	CONFIGURE	Set the security level as directed by faculty.	DVWA Security	Level = Low/Medium

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

Environment check (DVWA)

Component	Setting	Status
Web server	Apache on 127.0.0.1	Running
Database	MySQL (dvwa schema)	Connected
Exposure	Localhost only	Safe
Security level	Low	Set

The app must be reachable only from localhost. Never expose a vulnerable application to a shared or public network.

Reusable Template

Environment record template

Component	Setting	Verified? (Y/N)
Web server		
Database		



Component	Setting	Verified? (Y/N)
Exposure		
Security level		

Your Output / Screenshot (paste result here):



Q2

Perform a basic SQL injection test

5 Marks

Objective: Show how unsanitised input lets an attacker alter a database query, and record the impact.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	Find an input field that reaches a database query.	DVWA SQLi page	Target input
2	TEST	Submit a normal value and note the response.	e.g. 1	Baseline behaviour
3	EXECUTE	Submit a classic injection payload.	1' OR '1'='1	Extra rows returned
4	ANALYSE	Observe what data is exposed.	—	Unexpected records
5	REPORT	Record the payload, response and potential impact.	Notes	Finding documented

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

SQL injection result (DVWA)

Input submitted	App behaviour	Impact
1	Returns one user	Normal
1' OR '1'='1	Returns all users	Data exposed

The always-true condition makes the query return every row, showing how injection can dump an entire table.

Reusable Template

Test inputs (use against DVWA only)

```
1' OR '1'='1
1' UNION SELECT user, password FROM users -- -
```

Your Output / Screenshot (paste result here):





Q3

Simulate an XSS attack and show a mitigation

5 Marks

Objective: Trigger a cross-site scripting payload, then show that encoding or validation stops it.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	Find a field whose input is reflected on the page.	DVWA XSS page	Reflected input
2	EXECUTE	Submit a script payload.	<script>alert('xss')</script>	Alert box appears
3	ANALYSE	Confirm the script executed in the browser.	—	JavaScript ran
4	CONFIGURE	Apply output encoding or input validation.	htmlspecialchars()	Payload neutralised
5	TEST	Re-submit the payload to confirm the fix.	re-test	Script shown as text

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

XSS before and after mitigation

State	Input	Result in browser
Vulnerable	<script>alert('xss')</script>	Alert pops up
Mitigated	<script>alert('xss')</script>	Shown as plain text

After encoding, the browser treats the payload as text instead of code, so the script no longer runs.

Reusable Template

Payload and mitigation

```
<script>alert('xss')</script>      # payload (DVWA only)
echo htmlspecialchars($input);    # PHP output encoding fix
```

Your Output / Screenshot (paste result here):



K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION



Q4

Inspect requests and responses with OWASP ZAP / Burp

5 Marks

Objective: Route traffic through an intercepting proxy to inspect requests, responses and flagged issues.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	Start the proxy and set the browser to use it.	OWASP ZAP / Burp	Proxy listening
2	EXECUTE	Browse the target so traffic flows through the proxy.	browse DVWA	Requests captured
3	ANALYSE	Inspect a request and its parameters.	request view	Headers + params
4	ANALYSE	Inspect the matching response.	response view	Status + body
5	TEST	Review any automated alerts the tool raised.	alerts tab	Issue list

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

Proxy inspection (DVWA)

Item	Example observed	Why it matters
Request parameter	id=1	Entry point for injection
Response code	200 OK	Request succeeded
Tool alert	Reflected XSS	Investigate

The proxy makes the hidden request/response detail visible, and its alerts point you to likely vulnerabilities to verify by hand.

Reusable Template

Typical proxy setup

Proxy listen address : 127.0.0.1
Proxy port : 8080
Browser proxy : 127.0.0.1:8080



Your Output / Screenshot (paste result here):



Q5

Analyse logs to find traces of the simulated attacks

5 Marks

Objective: Search server and application logs for the requests generated by your SQL injection and XSS tests.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	Locate the web server access log.	<code>/var/log/apache2/access.log</code>	Log file found
2	EXECUTE	Watch the log live while repeating a test.	<code>sudo tail -f access.log</code>	Requests appear
3	ANALYSE	Search the log for injection keywords.	<code>grep -Ei 'union select'</code>	SQLi lines
4	ANALYSE	Search the log for script payloads.	<code>grep -Ei '<script>'</code>	XSS lines
5	REPORT	Record the matching log entries as evidence.	Notes	Evidence saved

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

Log evidence (DVWA)

Attack	Log search	Sample match
SQL injection	<code>grep -Ei 'union select'</code>	GET /vuln?id=1' UNION SELECT...
XSS	<code>grep -Ei '<script>'</code>	GET /xss?name=<script>alert...

The log lines are timestamped proof that the attacks reached the server, which is exactly what an analyst uses during incident response.

Reusable Template

Log-analysis commands

```
sudo tail -f /var/log/apache2/access.log
grep -Ei "union|select|<script>" /var/log/apache2/access.log
```

Your Output / Screenshot (paste result here):





Q6

Apply remediation and verify the fix

5 Marks

Objective: Fix at least one finding using a proper control, then re-test to prove the vulnerability is closed.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	Pick a confirmed finding from Q2-Q3.	Q2 / Q3 finding	Target chosen
2	CONFIGURE	Apply the appropriate control.	prepared statements / encoding / header	Fix applied
3	EXECUTE	Restart the service if required.	sudo systemctl restart apache2	Service reloaded
4	TEST	Re-run the original attack.	re-test	Attack fails
5	REPORT	Record before-and-after evidence of the fix.	Screenshots	Fix verified

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

Remediation result (DVWA)

Finding	Control applied	Re-test result
SQL injection	Prepared statements	Blocked
XSS	Output encoding	Blocked
Missing headers	Add CSP header	Present

Each fix is proven by re-running the exact attack and showing it no longer works, which is what makes remediation credible.

Reusable Template

Example security header (Apache)

```
Header always set X-Content-Type-Options "nosniff"
Header always set Content-Security-Policy "default-src 'self'"
```

Your Output / Screenshot (paste result here):





Q7

Prepare a vulnerability report for each finding

10 Marks

Objective: Document every finding with its attack, log evidence, impact and verified remediation.

Step-by-Step Algorithm

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	Collect findings and evidence from Q2-Q6.	Q2-Q6 outputs	Evidence gathered
2	REPORT	For each finding, write attack, impact and fix.	Report	Finding entries
3	REPORT	Attach proxy captures, log lines and screenshots.	Evidence	Evidence attached
4	ANALYSE	Rate each finding's severity.	severity scale	Severity set
5	REPORT	Review, export and push the report to GitHub.	PDF + GitHub	Final deliverable

Phase key: ● SETUP · ● CONFIGURE · ● EXECUTE · ● TEST · ● ANALYSE · ● REPORT · ● FINISH

Worked Example (with sample values & output)

Vulnerability report summary (DVWA)

Finding	Severity	Evidence	Remediation
SQL injection	HIGH	Log + screenshot	Prepared statements
Reflected XSS	MEDIUM	Proxy capture	Output encoding

A good report lets someone else reproduce the finding from your evidence and confirm the fix works.

Reusable Template

Report submission checklist

✓	Check before submitting
☐	Every finding has attack, evidence, impact and fix
☐	Severity rated for each finding
☐	Re-test proof included for remediated items
☐	Report + GitHub link uploaded to ZOSIMA



Your Output / Screenshot (paste result here):



Lab 4 Assessment Scheme

Assessment	Basis	Marks
Question 1	Set up DVWA or an equivalent vulnerable app in an isolated environment.	5
Question 2	Perform a basic SQL injection test and record behaviour and impact.	5
Question 3	Simulate an XSS attack and demonstrate a mitigation.	5
Question 4	Use OWASP ZAP or Burp to inspect requests, responses and indicators.	5
Question 5	Analyse logs to identify traces of the simulated attacks.	5
Question 6	Apply remediation and verify the fix.	5
Question 7	Prepare a vulnerability report covering attack, evidence, impact and remediation.	10
Lab Assessment on ZOSIMA	Live four-criteria rubric (efforts & execution, output & correctness, testing & verification, explanation & authenticity), verified via GitHub.	40
Total		100



Lab 5: Capstone - Designing and Defending a Small Business Network

Problem	Design and defend a small business network by combining threat mapping, system hardening, encryption, web vulnerability testing and incident response. Demonstrate realistic security controls, logged evidence and a structured defence report.
Submission	Submit both an accessible GitHub/project link and a complete capstone document on ZOSIMA. The repository should contain organised code/scripts, regular commits, a README and sample logs.

Capstone Approach (Algorithm)

Step	Phase	Action	Command / Tool	What You Should See
1	SETUP	Choose a small business scenario; list assets, users and data.	Scenario brief	Asset & user list
2	ANALYSE	Identify the threats and vulnerabilities for that scenario.	ATT&CK / OWASP	Threat list
3	CONFIGURE	Design a segmented network with a firewall boundary.	Network diagram	Labelled topology
4	CONFIGURE	Plan controls: firewall, hardening, encryption, access.	Control plan	Control checklist
5	EXECUTE	Implement the controls in a virtual environment.	VM lab	Working controls
6	EXECUTE	Simulate attacks from Labs 1-4 against the setup.	SQLi / XSS / scan	Attacks logged
7	ANALYSE	Capture logs/alerts and show detection or blocking.	Logs / alerts	Detection evidence
8	TEST	Test valid, invalid and boundary cases; tune controls.	Test cases	Improved posture
9	REPORT	Write the report and organise the GitHub repository.	Report + GitHub	Complete deliverable



Phase key: • **SETUP** • • **CONFIGURE** • • **EXECUTE** • • **TEST** • • **ANALYSE** • • **REPORT** • • **FINISH**

Capstone Evaluation (200 Marks)

Phase	Focus	Marks
Phase 1 - Threat Analysis & Network Design	Problem clarity, threat understanding, network design	40
Phase 2 - Implementation & Security Controls	Implementation, execution, working controls, integration	40
Phase 3 - Attack Simulation & Detection	Output correctness, log evidence, detection accuracy	40
Phase 4 - Testing, Validation & Optimisation	Testing quality, validation, optimisation, reliability	40
Phase 5 - Results, Documentation & Presentation	Explanation, documentation, originality, presentation	40
Total		200

Viva and Final Submission

Component	Focus	Marks
Viva 1	Lab 1 and Lab 2: threat mapping, breach analysis, MITRE ATT&CK, Linux hardening.	100
Viva 2	Lab 3, Lab 4 and capstone: encryption, PKI, web security, OWASP, log analysis, defence.	100
Final Lab File Upload	Consolidated submission of all lab records, algorithms, reports and evidence on ZOSIMA.	50

— End of Lab Manual —